BÁO CÁO LAB 01 HỆ THỐNG LINUX NHÚNG ARMV7

Cấu hình Kernel và khởi động hệ thống trên QEMU vexpress a9

Linux Kernel 5.15 U-Boot 2022.04 BusyBox 1.35.0

Name: Trần Thiên Phúc

ID Student: SE203798

Lớp/Nhóm: IC2008

Ngày thực hiện: 25/09/2026

1 Tổng quan bài thực hành

Bài LAB01 xây dựng một hệ thống Linux nhúng tối giản cho kiến trúc ARMv7-A Cortex-A9. Toàn bộ hệ thống được cross-compile trên máy Ubuntu x86_64 và chạy trên QEMU với board vexpress-a9. Kết quả cuối cùng là hệ thống có thể khởi động từ Linux kernel, nhận Device Tree, giải nén initramfs, chạy BusyBox init và đưa người dùng tới dấu nhắc lệnh / #.

Điểm quan trọng của bài lab không chỉ là chạy được QEMU. Qua từng bước, người thực hiện hiểu vai trò của bốn thành phần chính: U-Boot chuẩn bị môi trường boot; Linux kernel quản lý CPU, bộ nhớ và thiết bị; Device Tree mô tả phần cứng; còn BusyBox cùng initramfs tạo userspace tối giản. Khi ghép đúng bốn phần này, một hệ thống Linux nhúng hoàn chỉnh có thể hoạt động dù không có ổ đĩa thật.

1.1 Mục tiêu

- Thiết lập cross-toolchain ARM hard-float và phân biệt máy host với target ARM.
- Cấu hình, biên dịch Linux kernel 5.15 và Device Tree cho QEMU vexpress-a9.
- Biên dịch U-Boot 2022.04, kiểm tra thông tin board và thiết lập bootargs, bootcmd.
- Biên dịch BusyBox 1.35.0 theo kiểu static linking và tạo root filesystem tối giản.
- Đóng gói root filesystem thành initramfs rồi boot tới BusyBox shell.
- Kiểm tra hostname, số CPU online, device nodes và các filesystem ảo sau khi boot.

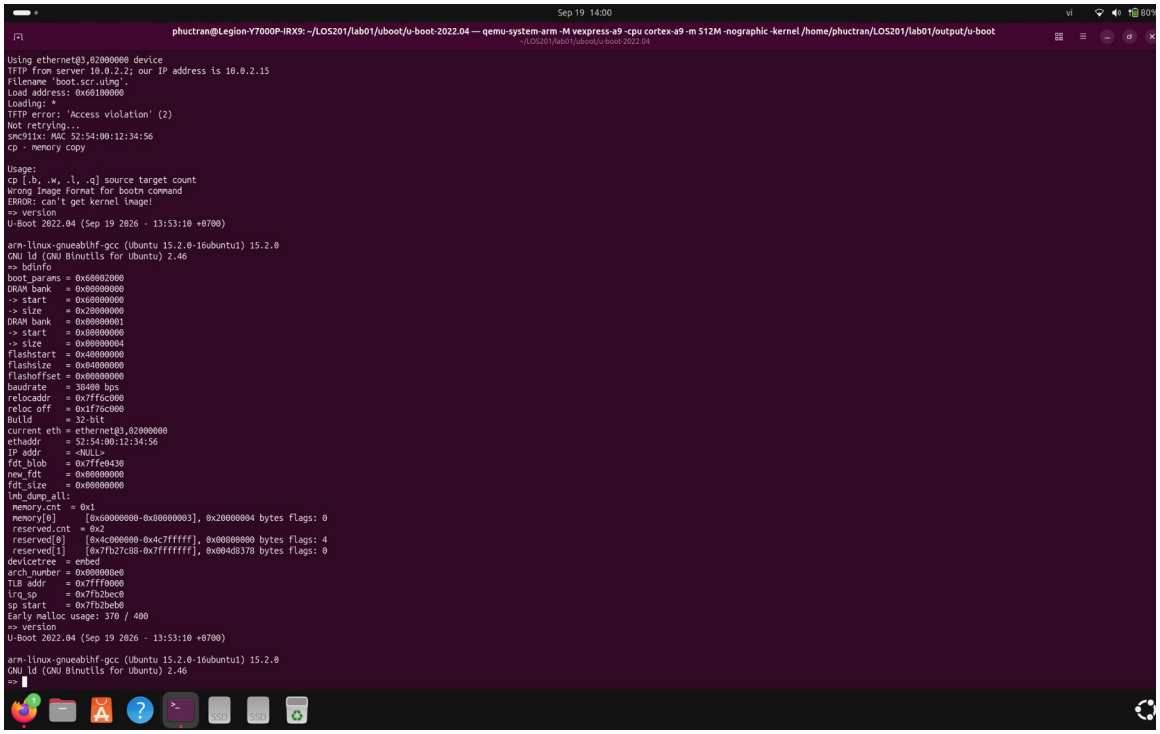
1.2 Kết quả đạt được

Thành phần	Kết quả thực tế	Ý nghĩa
Linux kernel	zImage khoảng 4.9 MB	Kernel ARMv7 đã được biên dịch thành công
Device Tree	vexpress-v2p-ca9.dtb khoảng 14 KB	Mô tả phần cứng của board vexpress-a9
U-Boot	u-boot khoảng 4.2 MB	Bootloader chạy được trong QEMU
Initramfs	rootfs-final.cpio.gz khoảng 1.1 MB	Userspace BusyBox tối giản, nạp hoàn toàn vào RAM
SMP	CPU online là 0-1	Hai CPU ảo hoạt động theo tùy chọn -smp 2
Userspace	Hostname embedded-lab, /tmp là tmpfs	Script rcS và cấu hình rootfs hoạt động đúng

1.3 Cấu trúc thư mục làm việc thực tế

Trong quá trình thực hiện, toàn bộ mã nguồn và sản phẩm được tổ chức dưới thư mục /home/phuctran/LOS201/lab01. Cách chia thư mục giúp tách mã nguồn, root filesystem, file cấu hình và sản phẩm cuối, nhờ đó dễ kiểm tra và đóng gói bài nộp.

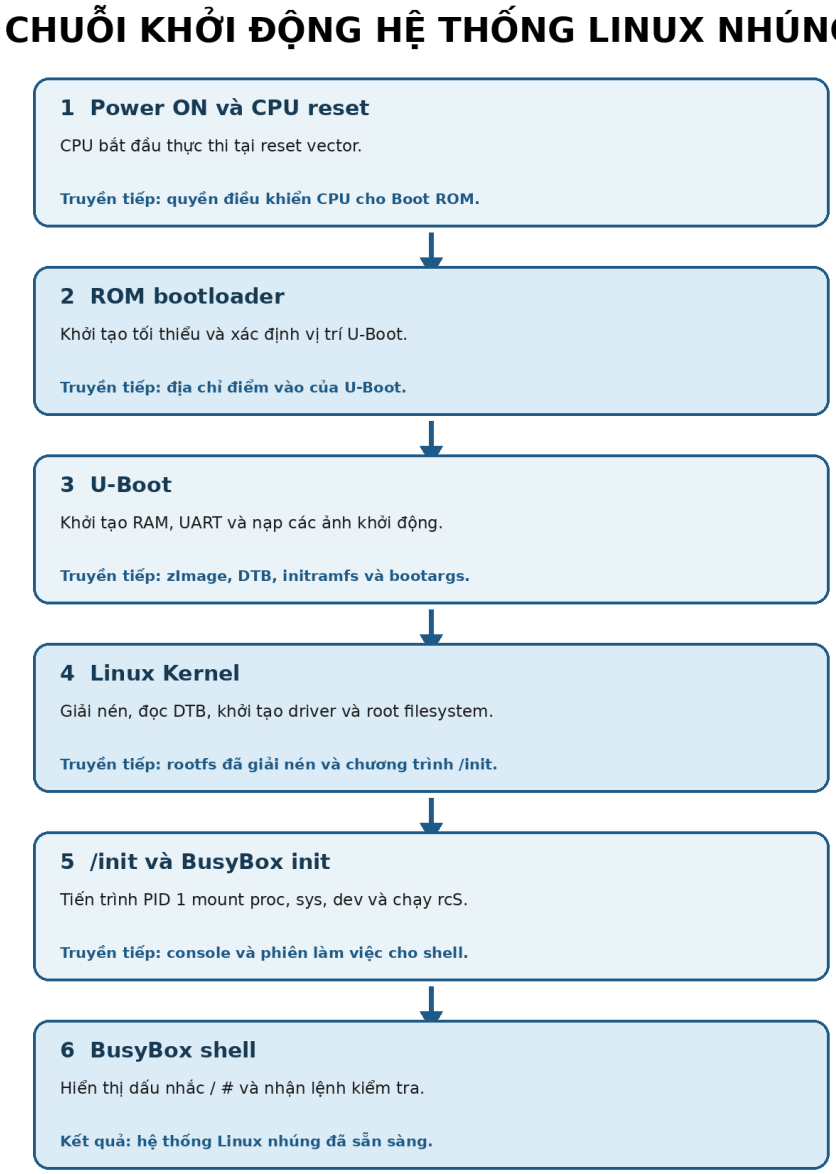
```
/home/phuctran/LOS201/lab01/
├── kernel/linux-5.15/
├── uboot/u-boot-2022.04/
├── busybox/busybox-1.35.0/
├── rootfs/
├── configs/
│   ├── kernel-5.15.config
│   └── busybox-1.35.0.config
└── output/
    ├── zImage
    ├── vexpress-v2p-ca9.dtb
    ├── u-boot
    └── rootfs-final.cpio.gz
```



Hình 1. Kiểm tra cross-toolchain ARM và QEMU trên máy Ubuntu host

2 Sơ đồ chuỗi khởi động

Sơ đồ dưới đây trình bày chuỗi boot từ lúc cấp nguồn đến BusyBox shell: Power ON, ROM bootloader, U-Boot, Linux kernel, tiến trình /init và BusyBox shell. Initramfs được thể hiện là dữ liệu U-Boot chuyển cho kernel, không phải một thành phần tự thực thi. Mỗi khối nêu thành phần đang chạy, công việc chính và dữ liệu được chuyển sang bước kế tiếp.



Hình 2. Chuỗi khởi động từ Power ON đến BusyBox shell

Lưu ý: Trong bài LAB, QEMU có thể nạp trực tiếp U-Boot hoặc kernel nên giai đoạn ROM bootloader không xuất hiện thành một dòng log riêng. Sơ đồ trên mô tả chuỗi khởi động logic đầy đủ theo yêu cầu của đề bài.

2.1 Giải thích ngắn gọn từng giai đoạn

Giai đoạn	Thành phần đang chạy	Công việc chính	Dữ liệu chuyển tiếp
1	Power ON và CPU reset	CPU bắt đầu thực thi tại reset vector	Quyền điều khiển CPU cho Boot ROM
2	ROM bootloader	Khởi tạo tối thiểu và xác định vị trí U-Boot	Địa chỉ điểm vào của U-Boot
3	U-Boot	Khởi tạo RAM, UART và nạp các ảnh khởi động	zImage, DTB, initramfs và bootargs
4	Linux kernel	Giải nén, đọc DTB, khởi tạo driver và giải nén initramfs thành rootfs	Rootfs đã sẵn sàng và chương trình /init
5	/init và BusyBox init	Chạy với PID 1, mount proc/sys/dev và thực thi rcS	Console và phiên làm việc cho shell
6	BusyBox shell	Hiện dấu nhắc / # và nhận lệnh kiểm tra	Hệ thống Linux nhúng sẵn sàng

3 Môi trường và công cụ

Hạng mục	Cấu hình sử dụng	Vai trò
Máy host	Ubuntu 64-bit	Nơi tải mã nguồn, cấu hình và biên dịch
Kiến trúc target	ARMv7-A Cortex-A9	Kiến trúc CPU của hệ thống nhúng mô phỏng
Board ảo	QEMU vexpress-a9	Mô phỏng Versatile Express Cortex-A9
Cross-compiler	arm-linux-gnueabihf-	Tạo binary ARM hard-float trên máy x86_64
Kernel	Linux 5.15 LTS	Quản lý phần cứng và cung cấp system call
Bootloader	U-Boot 2022.04	Thiết lập môi trường boot và tham số kernel
Userspace	BusyBox 1.35.0	Cung cấp init, shell và các lệnh Unix tối giản

Host là máy Ubuntu đang biên dịch chương trình. Target là máy ARMv7 được QEMU mô phỏng. Do hai máy có kiến trúc khác nhau, gcc thông thường của Ubuntu không phù hợp; phải dùng cross-compiler có tiền tố arm-linux-gnueabihf-.

4 Phần A Cấu hình và build Linux kernel

4.1 Thiết lập cross compilation

Ba biến môi trường được đặt trước khi chạy make. ARCH chọn kiến trúc đích ARM. CROSS_COMPILE yêu cầu hệ thống build dùng bộ công cụ ARM thay vì gcc của máy host. KCFLAGS bổ sung lựa chọn cho ARMv7-A, VFPv3 và hard-float theo cấu hình của bài lab.

```
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
export KCFLAGS="-march=armv7-a -mfpu=vfpv3 -mfloat-abi=hard"
```

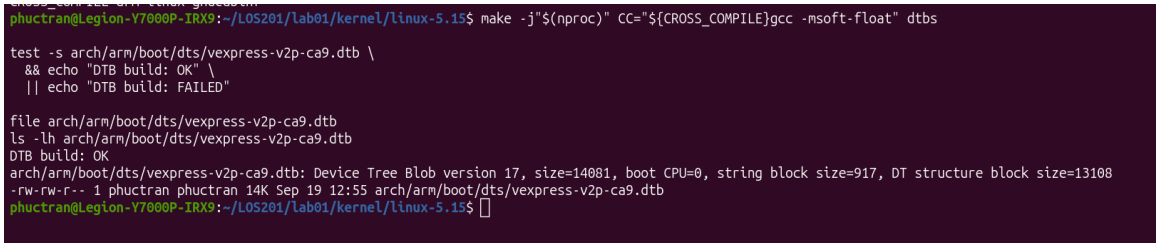
Lưu ý kỹ thuật: Linux kernel không dùng phép toán dấu phẩy động như chương trình userspace. Giá trị lớn nhất của việc chọn gnueabihf trong bài này là tạo một chuỗi công cụ thống nhất cho target ARMv7 hard-float, đặc biệt khi biên dịch BusyBox và userspace.

4.2 Nạp cấu hình mặc định và tùy chỉnh kernel

Cấu hình vexpress_defconfig cung cấp tập tùy chọn phù hợp với dòng ARM Versatile Express. Sau khi tạo .config, các mục quan trọng được kiểm tra hoặc bật gồm hỗ trợ platform VEXPRESS, CPU ARMv7, initramfs, serial PL011 và SMP.

```
cd /home/phuctran/LOS201/lab01/kernel/linux-5.15
make vexpress_defconfig
grep -E 'CONFIG_ARCH_VEXPRESS|CONFIG_CPU_V7|CONFIG_SMP' .config
make menuconfig
```

Tùy chọn	Lý do cần thiết
CONFIG_ARCH_VEXPRESS=y	Bật hỗ trợ board Versatile Express dùng bởi QEMU vexpress-a9
CONFIG_CPU_V7=y	Bật hỗ trợ tập lệnh và cơ chế của CPU ARMv7
Initramfs support	Cho phép kernel sử dụng root filesystem nằm trong RAM
ARM AMBA PL011	Cung cấp console ttyAMA0 để xem log qua -nographic
CONFIG_SMP=y	Cho phép kernel khởi tạo nhiều CPU, dùng với QEMU -smp 2



Hình 3. Các tùy chọn kernel quan trọng cho vexpress-a9

Minh chứng bổ sung theo đúng yêu cầu báo cáo: cấu hình versatile_defconfig được tạo trong một file KCONFIG_CONFIG riêng để không ghi đè cấu hình vexpress-a9 dùng cho lần build và boot thực tế.

```
/ # QEMU: Terminated
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab01$ cd /home/phuctran/LOS201/lab01/kernel/linux-5.15

CONFIG_FILE=/home/phuctran/LOS201/lab01/evidence/versatile_report.config

make KCONFIG_CONFIG="$CONFIG_FILE" versatile_defconfig

echo "===== VERSATILE DEFCONFIG RESULT ====="

grep -E 'CONFIG_ARCH_VERSATILE=|CONFIG_CPU_ARM926T=' \
"$CONFIG_FILE"
===== VERSATILE DEFCONFIG RESULT =====
CONFIG_ARCH_VERSATILE=y
CONFIG_CPU_ARM926T=y
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab01/kernel/linux-5.15$
```

Hình 4. Kết quả chạy make versatile_defconfig

Kết quả CONFIG_ARCH_VERSATILE=y và CONFIG_CPU_ARM926T=y xác nhận cấu hình mặc định dành cho nền tảng Versatile đã được tạo thành công.

4.3 Biên dịch kernel và Device Tree

Lệnh build tạo đồng thời kernel image nén zImage, các Device Tree Blob và module. Tùy chọn -j$(nproc) tận dụng nhiều CPU của máy host để rút ngắn thời gian biên dịch; nó không liên quan đến số CPU của máy ARM ảo.

```
make -j$(nproc) zImage dtbs modules
make modules_prepare

cp arch/arm/boot/zImage /home/phuctran/LOS201/lab01/output/
cp arch/arm/boot/dts/vexpress-v2p-ca9.dtb \
   /home/phuctran/LOS201/lab01/output/
```

zImage chứa kernel ARM đã nén và có đoạn mã tự giải nén. File vexpress-v2p-ca9.dtb không chứa chương trình; nó mô tả địa chỉ RAM, CPU, interrupt controller, UART và các thiết bị của board để kernel không phải hard-code toàn bộ phần cứng.

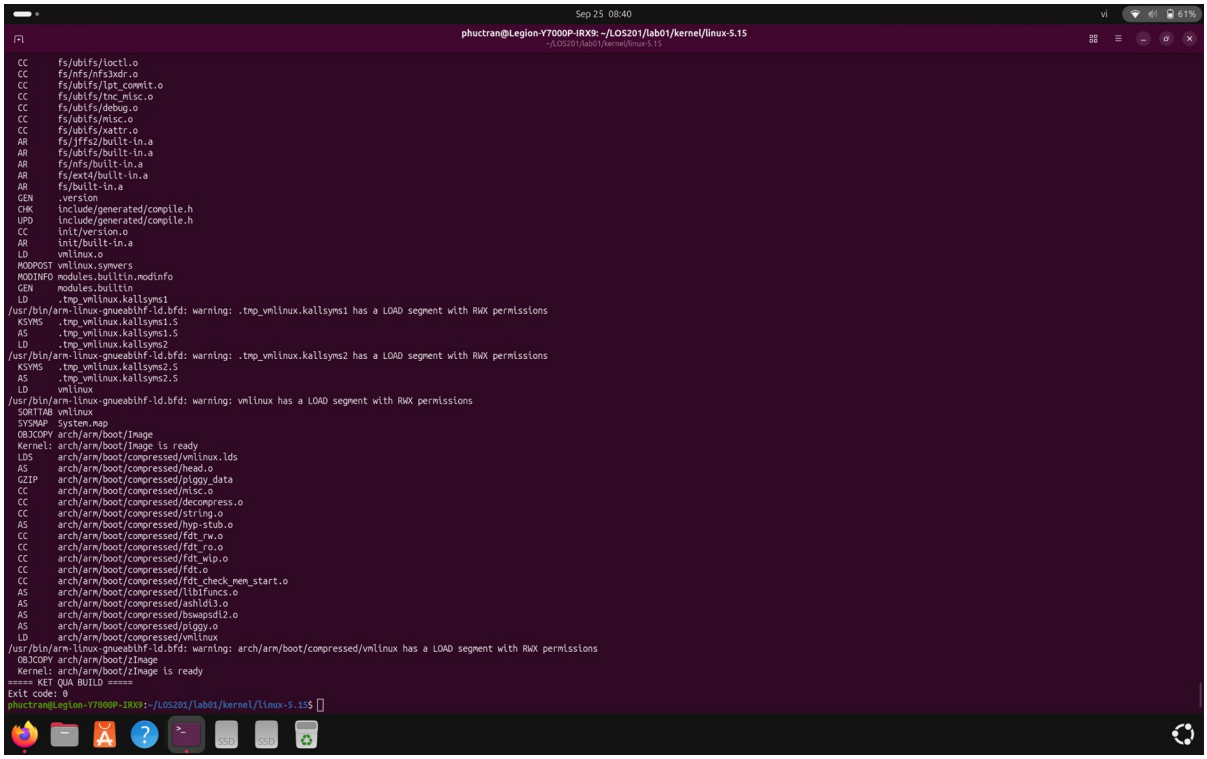
```
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab01/kernel/linux-5.15$ test -s /home/phuctran/LOS201/lab01/output/zImage \
  && echo "zImage: OK" \
  || echo "zImage: FAILED"

test -s /home/phuctran/LOS201/lab01/output/vexpress-v2p-ca9.dtb \
  && echo "DTB: OK" \
  || echo "DTB: FAILED"

ls -lh \
/home/phuctran/LOS201/lab01/output/zImage \
/home/phuctran/LOS201/lab01/output/vexpress-v2p-ca9.dtb
zImage: OK
DTB: OK
-rw-rw-r-- 1 phuctran phuctran  14K Sep 19 12:58 /home/phuctran/LOS201/lab01/output/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 phuctran phuctran 4.9M Sep 19 12:58 /home/phuctran/LOS201/lab01/output/zImage
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab01/kernel/linux-5.15$
```

Hình 5. Build Linux kernel và Device Tree thành công

Phần cuối của log build được lưu riêng để kiểm tra trạng thái hoàn tất. Mã thoát bằng 0 và thông báo zImage is ready cho thấy quá trình biên dịch không còn lỗi.



Hình 6. Log cuối quá trình build Linux kernel thành công



Hình 7. Kết quả ls -lh zImage và nhận dạng file kernel ARM

File zImage có kích thước khoảng 4.9 MB và được nhận dạng là Linux kernel ARM boot executable. Ba minh chứng trên hoàn tất yêu cầu của Phần A.

5 Phần B Cấu hình và sử dụng U Boot

5.1 Build U Boot cho Cortex A9

U-Boot được cấu hình bằng vexpress_ca9x4_defconfig vì QEMU vexpress-a9 mô phỏng board Versatile Express với Cortex-A9. Sau khi build, file u-boot là chương trình ELF 32-bit ARM EABI5 và được sao chép sang thư mục output.

```
cd /home/phuctran/LOS201/lab01/uboot/u-boot-2022.04
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make vexpress_ca9x4_defconfig
make -j$(nproc)
file u-boot
cp u-boot /home/phuctran/LOS201/lab01/output/
```

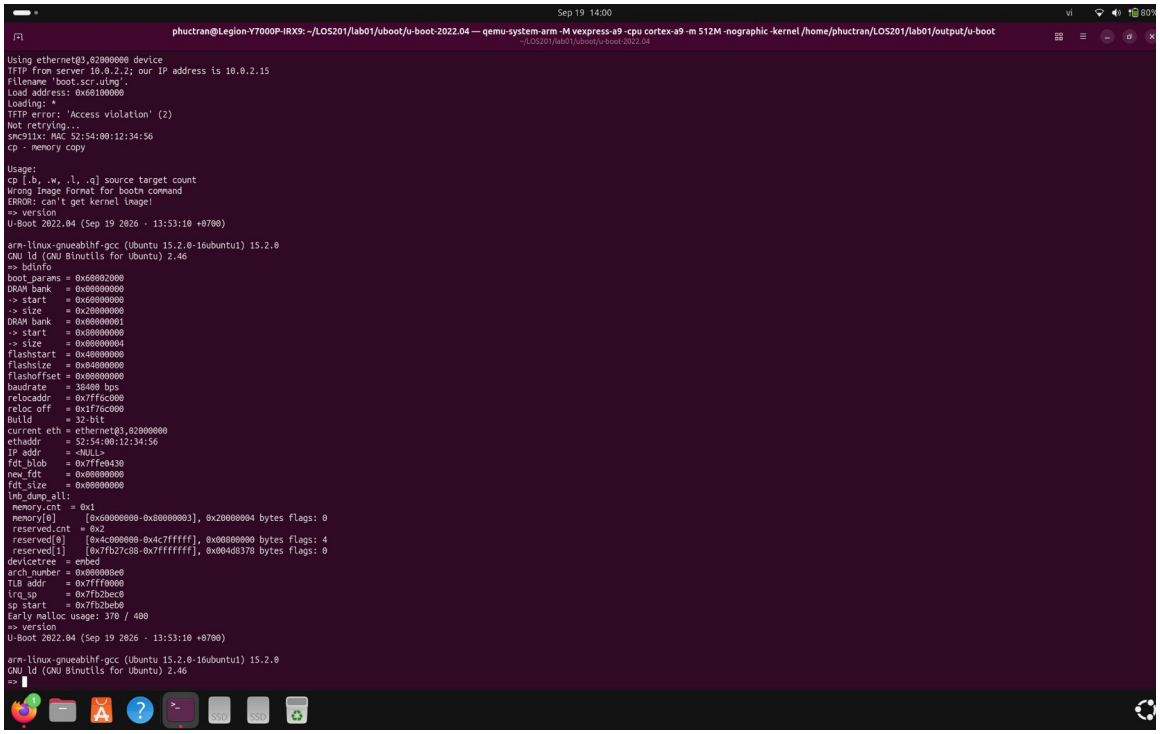


Hình 8. U-Boot 2022.04 được build đúng kiến trúc ARM

5.2 Khởi động vào U Boot shell

```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -nographic \
  -kernel /home/phuctran/LOS201/lab01/output/u-boot
```

QEMU nạp U-Boot vào máy ảo và nối UART PL011 với terminal hiện tại. Tùy chọn -nographic tắt cửa sổ đồ họa, vì vậy toàn bộ thông báo xuất hiện trực tiếp trong terminal. Khi thấy dấu nhắc =>, U-Boot đã chạy và sẵn sàng nhận lệnh.



Hình 9. Thông tin U-Boot và board vexpress-a9 trong QEMU

5.3 Thiết lập bootargs và bootcmd

bootargs là chuỗi tham số U-Boot truyền cho kernel. bootcmd là lệnh boot mặc định. Trong bài lab, bộ nhớ DRAM của vexpress-a9 bắt đầu tại 0x60000000 nên các địa chỉ nạp kernel, initramfs và DTB đều phải nằm trong vùng RAM này và không được chồng lấn.

```
setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init
mem=512M'
setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
printenv bootargs
printenv bootcmd
```

Tham số hoặc địa chỉ	Giải thích
console=ttyAMA0,115200	Dùng UART PL011 làm console với baud rate 115200
root=/dev/ram0	Root filesystem được đặt trong RAM
rw	Cho phép root filesystem ở chế độ đọc ghi
init=/sbin/init	Chọn tiến trình init sau khi kernel mount rootfs
mem=512M	Khớp dung lượng RAM được cấp bởi QEMU -m 512M
0x60100000	Địa chỉ dự kiến của zImage trong RAM
0x64000000	Địa chỉ dự kiến của initramfs trong RAM
0x60000000	Địa chỉ dự kiến của DTB trong RAM

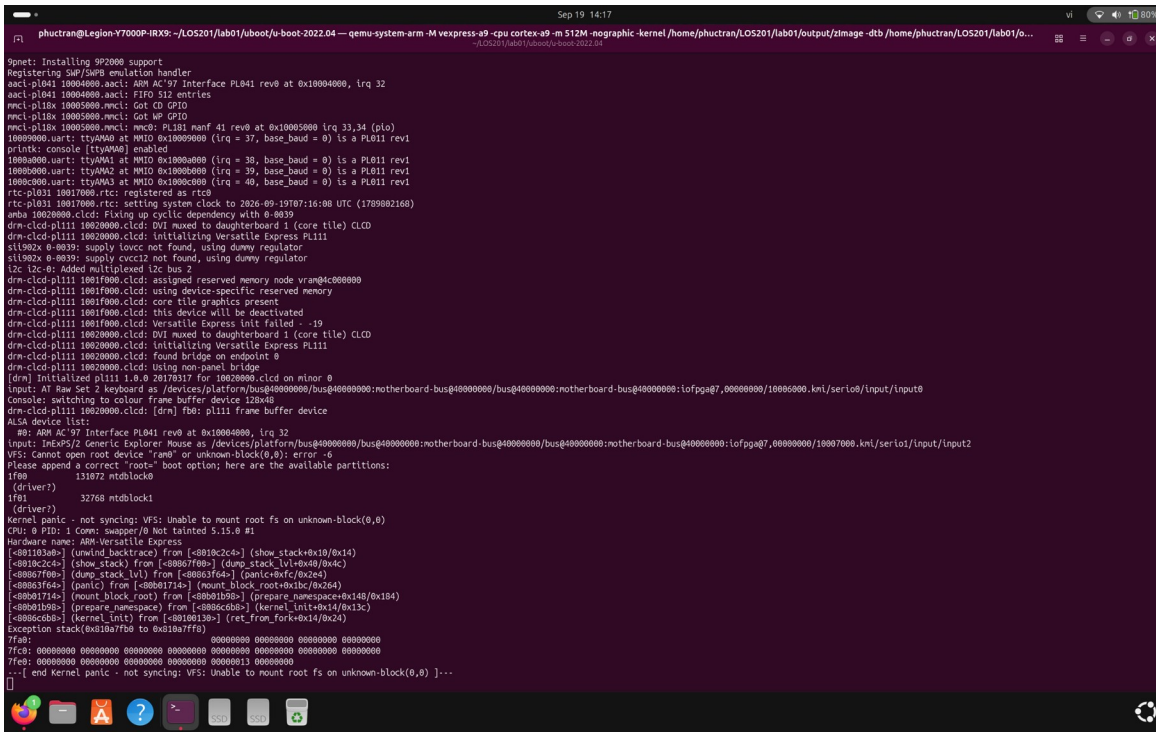
Trong QEMU, lệnh saveenv có thể cảnh báo do máy ảo không có vùng flash bền vững được cấu hình để lưu môi trường. Cảnh báo này không có nghĩa U-Boot build sai; các biến vẫn tồn tại trong phiên đang chạy.



Hình 10. Biến môi trường boot của U-Boot

5.4 Boot kernel khi chưa có root filesystem

Kernel được boot thử với zImage và DTB nhưng chưa cung cấp initramfs. Kernel vẫn khởi tạo CPU và driver, sau đó dừng ở bước mount root filesystem. Thông báo Kernel panic: VFS: Unable to mount root fs on unknown-block(1,0) là kết quả dự kiến, cho thấy phần kernel đã hoạt động nhưng userspace còn thiếu.



Hình 11. Kernel panic dự kiến khi chưa có initramfs

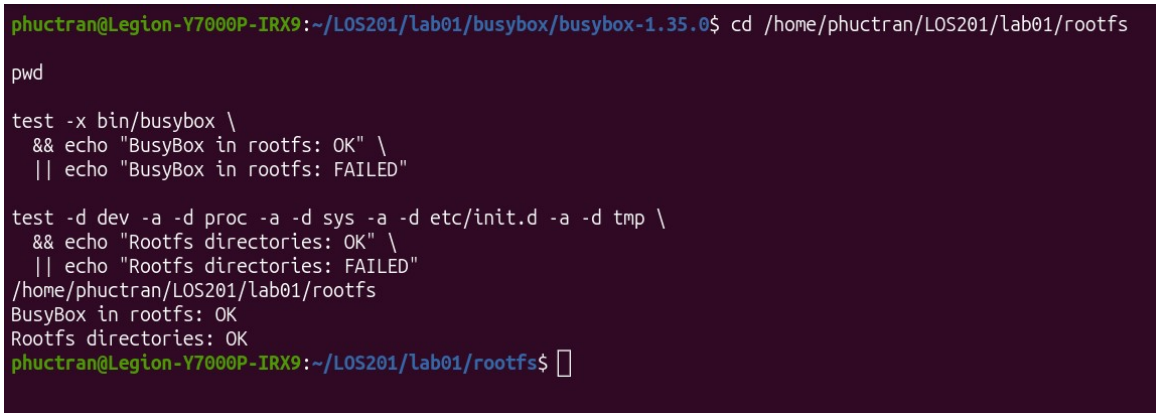
6 Phần C Xây dựng root filesystem và initramfs

6.1 Build BusyBox theo kiểu static linking

BusyBox gom nhiều tiện ích Unix vào một executable duy nhất. Static linking đưa các phần thư viện cần thiết vào chính file busybox, vì vậy rootfs không cần mang theo dynamic loader và các shared library. Đây là lựa chọn phù hợp cho initramfs nhỏ và dễ boot.

```
cd /home/phuctran/LOS201/lab01/busybox/busybox-1.35.0
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
make defconfig
make menuconfig
make -j$(nproc)
make install
file _install/bin/busybox
```

Kết quả kiểm tra phải cho thấy ELF 32-bit ARM, EABI5 và statically linked. Thư mục _install chứa BusyBox cùng các symbolic link như sh, mount, hostname và ps; các link này đều gọi về cùng một binary busybox.



Hình 12. BusyBox ARM được build static và cài vào thư mục _install

6.2 Tạo cấu trúc root filesystem

Root filesystem tối giản vẫn phải có các thư mục đúng vai trò. /bin và /sbin chứa lệnh; /etc chứa cấu hình; /proc và /sys là điểm mount để kernel xuất thông tin; /dev chứa device nodes; /tmp lưu dữ liệu tạm; /root là home của tài khoản root.

```
rootfs/
├── bin/          ├── sbin/
├── usr/bin/      ├── usr/sbin/
├── etc/init.d/   ├── root/
├── dev/          ├── proc/
├── sys/          └── tmp/
```

6.3 Các file khởi tạo quan trọng

File /init là entry point khi QEMU truyền rdinit=/init. Nó không cần lặp lại toàn bộ công việc khởi tạo mà chuyển quyền điều khiển sang BusyBox init. Sau đó /sbin/init đọc /etc/inittab, chạy /etc/init.d/rcS và cuối cùng mở shell trên console.

```
#!/bin/sh
exec /sbin/init
```

Nội dung chính của /etc/init.d/rcS được hoàn thiện như sau:

```
#!/bin/sh

mount -t proc proc /proc
mount -t sysfs sysfs /sys
mount -t devtmpfs devtmpfs /dev
mount -t tmpfs tmpfs /tmp
chmod 1777 /tmp

hostname "$(cat /etc/hostname)"
echo "Welcome to LAB01 Embedded Linux"
```

Lệnh trong rcS	Tác dụng
mount -t proc	Tạo /proc để xem tiến trình, CPU và thông tin kernel
mount -t sysfs	Tạo /sys để truy cập mô hình thiết bị và thuộc tính kernel
mount -t devtmpfs	Để kernel tạo device nodes trong /dev
mount -t tmpfs	Tạo /tmp trong RAM; dữ liệu mất khi tắt máy
chmod 1777 /tmp	Cho phép mọi tiến trình dùng /tmp nhưng giữ sticky bit
hostname ...	Đọc embedded-lab từ /etc/hostname và áp dụng khi boot

Ngoài ra, /etc/passwd có dòng root:x:0:0:root:/root:/bin/sh để khai báo tài khoản root, còn /etc/hostname chứa embedded-lab. Các file này giúp hệ thống có thông tin người dùng và tên máy rõ ràng dù userspace rất nhỏ.

6.4 Đóng gói initramfs

Toàn bộ cây thư mục rootfs được đóng gói theo định dạng cpio newc rồi nén bằng gzip. Tùy chọn --owner=0:0 đảm bảo mọi file trong archive thuộc root khi chạy trên target, bất kể UID và GID của người dùng trên máy host.

```
cd /home/phuctran/LOS201/lab01/rootfs
find . -print0 \
  | cpio --null -o --format=newc --owner=0:0 \
  | gzip -9 \
  > /home/phuctran/LOS201/lab01/output/rootfs-final.cpio.gz

gzip -t /home/phuctran/LOS201/lab01/output/rootfs-final.cpio.gz
```

Kết quả thực tế là 3199 blocks, gzip kiểm tra thành công với thông báo Final initramfs: OK, và file rootfs-final.cpio.gz có kích thước khoảng 1.1 MB.



Hình 13. Đóng gói và kiểm tra initramfs hoàn chỉnh

Archive rootfs-final.cpio.gz được giải nén vào thư mục kiểm tra riêng. Việc liệt kê nội dung giúp xác nhận initramfs có đủ cấu trúc tối thiểu.

```
./output/rootfs.cpio.gz
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab01$ cd /home/phuctran/LOS201/lab01

mkdir -p evidence/initramfs

cd evidence/initramfs

gzip -dc ../../output/rootfs-final.cpio.gz | cpio -idmu

cd /home/phuctran/LOS201/lab01

echo "===== NOI DUNG INITRAMFS ====="

ls -lah evidence/initramfs/
3199 blocks
===== NOI DUNG INITRAMFS =====
total 48K
drwxrwxr-x 11 phuctran phuctran 4.0K Sep 25 08:46 .
drwxrwxr-x  4 phuctran phuctran 4.0K Sep 25 08:46 ..
drwxrwxr-x  2 phuctran phuctran 4.0K Sep 25 08:46 bin
drwxrwxr-x  2 phuctran phuctran 4.0K Sep 19 15:04 dev
drwxrwxr-x  3 phuctran phuctran 4.0K Sep 25 08:46 etc
-rwxrwxr-x  1 phuctran phuctran   27 Sep 22 09:09 init
lrwxrwxrwx  1 phuctran phuctran   11 Sep 19 14:58 linuxrc -> bin/busybox
drwxrwxr-x  2 phuctran phuctran 4.0K Sep 19 15:04 proc
drwxrwxr-x  2 phuctran phuctran 4.0K Sep 22 09:11 root
drwxrwxr-x  2 phuctran phuctran 4.0K Sep 25 08:46 sbin
drwxrwxr-x  2 phuctran phuctran 4.0K Sep 19 15:04 sys
drwxrwxrwt  2 phuctran phuctran 4.0K Sep 19 15:04 tmp
drwxrwxr-x  4 phuctran phuctran 4.0K Sep 25 08:46 usr
phuctran@Legion-Y7000P-IRX9:~/LOS201/lab01$
```

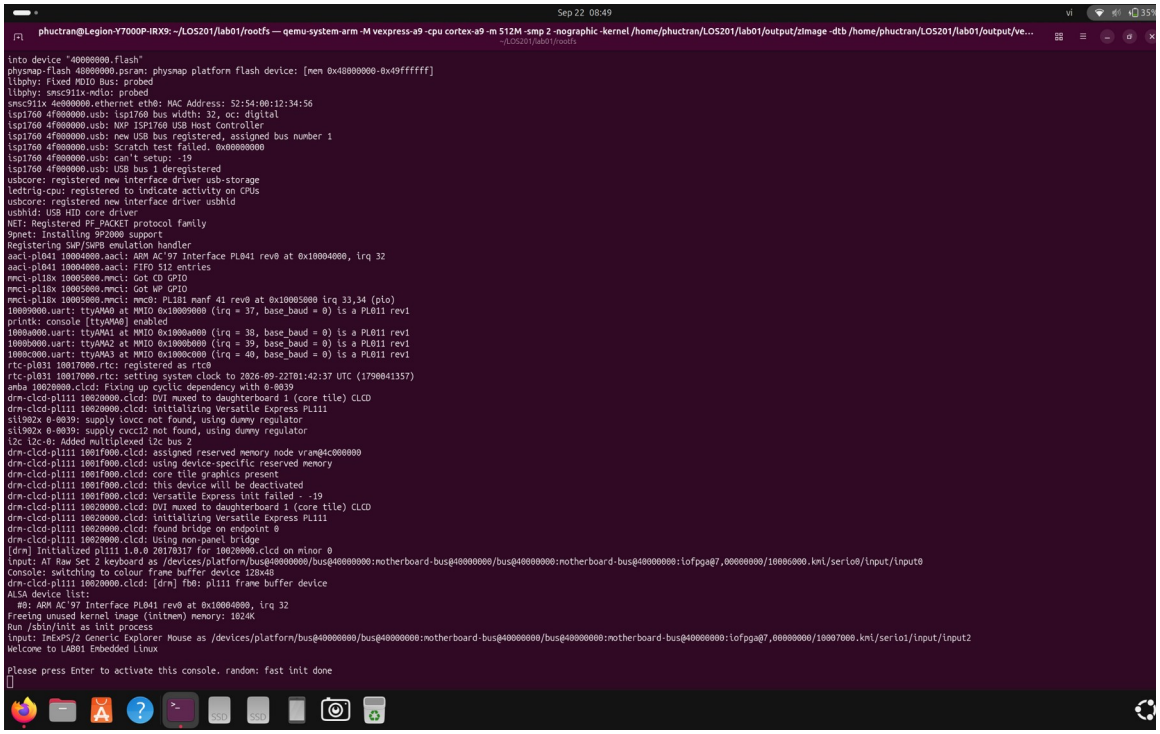
Hình 14. Kết quả liệt kê nội dung thư mục initramfs

Các thư mục bin, dev, etc, proc, sbin, sys, tmp và usr cùng file /init cho thấy root filesystem tối giản đã đủ thành phần để kernel chuyển sang userspace.

6.5 Boot hệ thống hoàn chỉnh trên QEMU

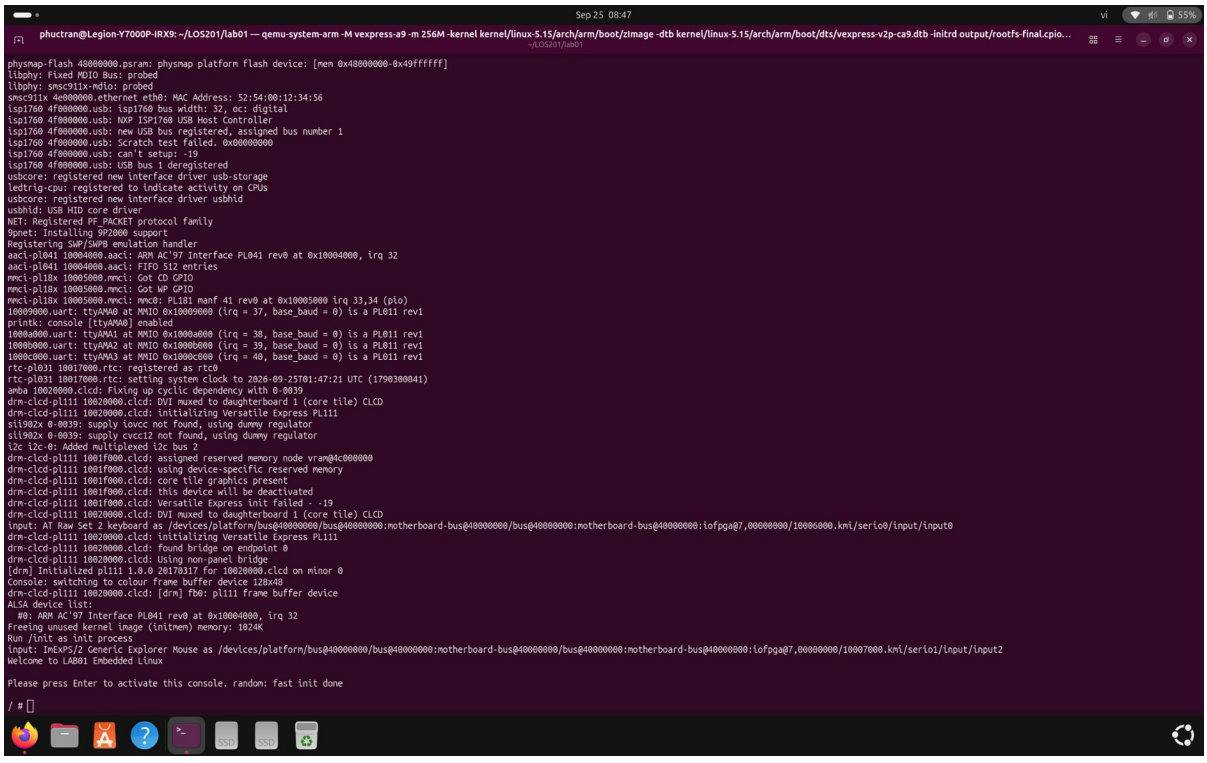
```
qemu-system-arm \
  -M vexpress-a9 \
  -cpu cortex-a9 \
  -m 512M \
  -smp 2 \
  -nographic \
  -kernel /home/phuctran/LOS201/lab01/output/zImage \
  -dtb /home/phuctran/LOS201/lab01/output/vexpress-v2p-ca9.dtb \
  -initrd /home/phuctran/LOS201/lab01/output/rootfs-final.cpio.gz \
  -append "console=ttyAMA0,115200 rdinit=/init mem=512M"
```

Ở lần boot cuối, kernel nhận đủ zImage, DTB và initramfs. Tham số rdinit=/init yêu cầu kernel chạy script /init vừa tạo. Tùy chọn -smp 2 tạo hai CPU ảo. Khi thấy Welcome to LAB01 Embedded Linux và dấu nhắc / #, chuỗi boot đã hoàn thành.



Hình 15. Hệ thống Linux nhúng boot thành công tới BusyBox shell

Ảnh boot kiểm tra lại bằng zImage mới cho thấy kernel chạy /init, hiển thị thông báo chào của LAB01 và trả về dấu nhắc / # của BusyBox shell.



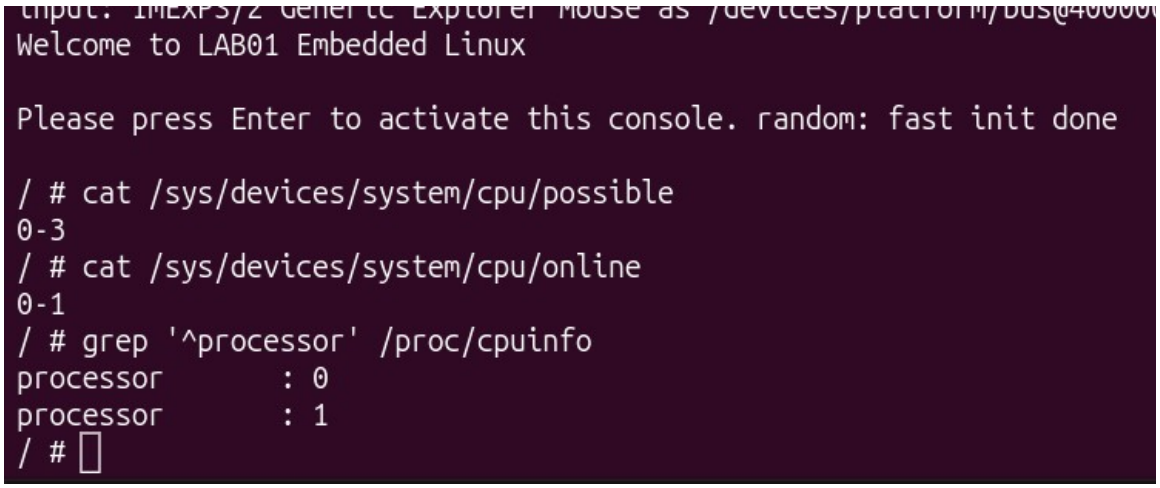
Hình 16. QEMU boot thành công tới BusyBox shell

6.6 Xác minh hệ thống sau khi boot

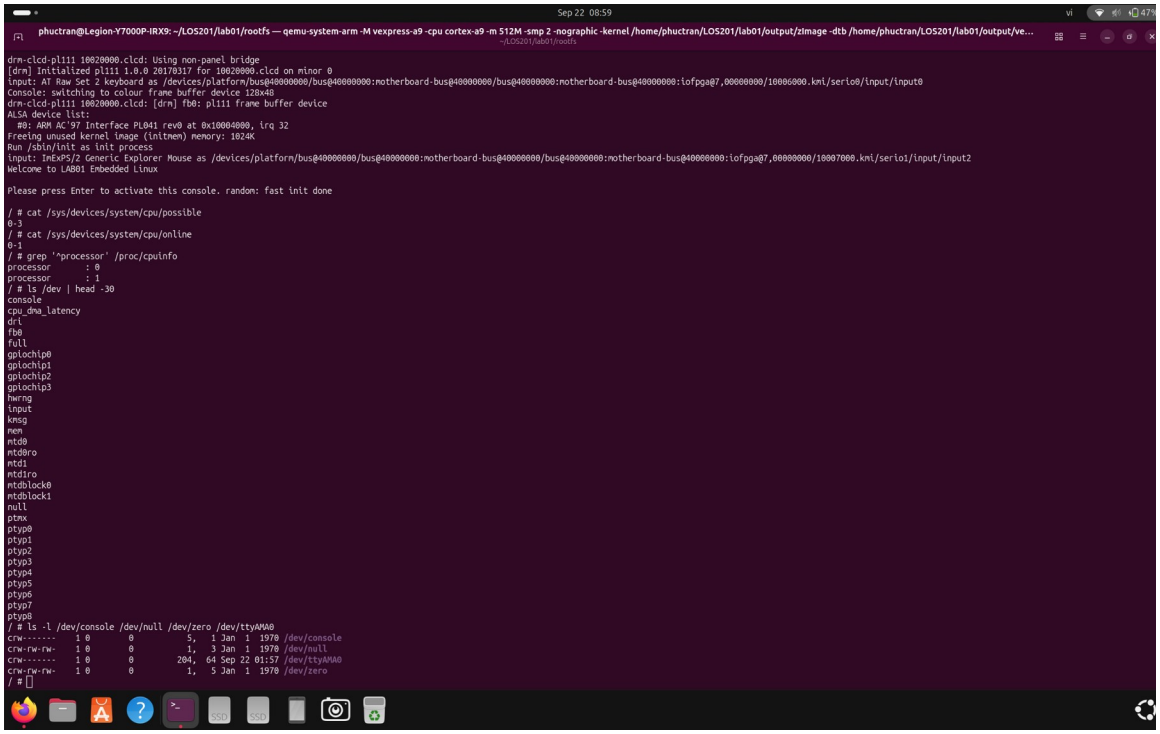
Các lệnh kiểm tra sau chứng minh kernel và userspace không chỉ boot được mà còn nhận đúng CPU, device nodes và filesystem ảo.

```
cat /sys/devices/system/cpu/possible
cat /sys/devices/system/cpu/online
grep '^processor' /proc/cpuinfo
ls /dev | head -30
ls -l /dev/console /dev/null /dev/zero /dev/ttyAMA0
hostname
uname -n
mount | grep ' /tmp '
```

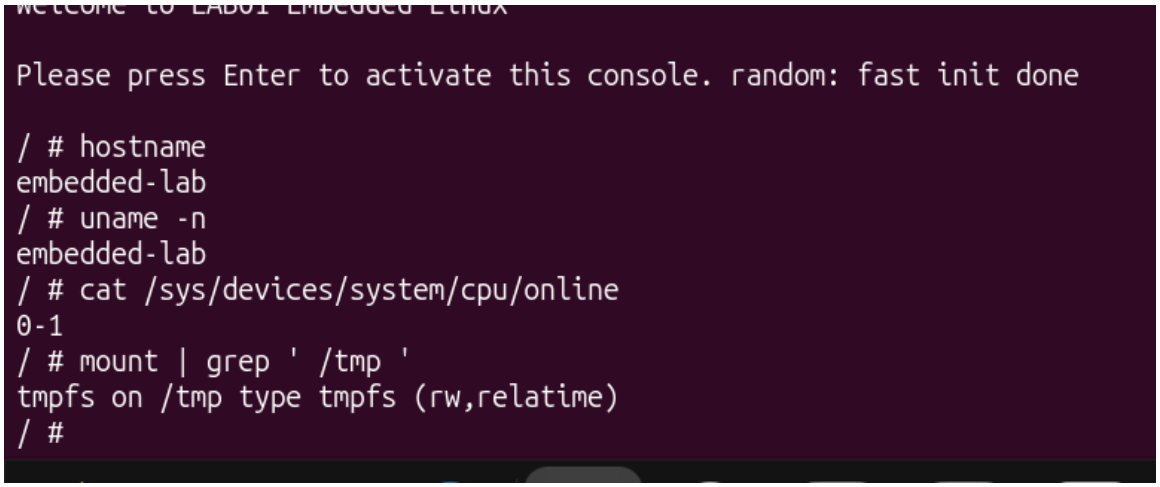
Kiểm tra	Kết quả thực tế	Kết luận
CPU possible	0-3	Platform có thể mô phỏng tối đa bốn CPU
CPU online	0-1	Hai CPU đang hoạt động theo -smp 2
processor	processor 0 và processor 1	/proc/cpuinfo nhận đủ hai CPU
Device nodes	console, null, zero, ttyAMA0 đều tồn tại	devtmpfs hoạt động đúng
hostname và uname -n	embedded-lab	rcS áp dụng /etc/hostname thành công
Mount /tmp	tmpfs on /tmp type tmpfs	/tmp được lưu trong RAM



Hình 17. Xác minh hai CPU Cortex-A9 đang online

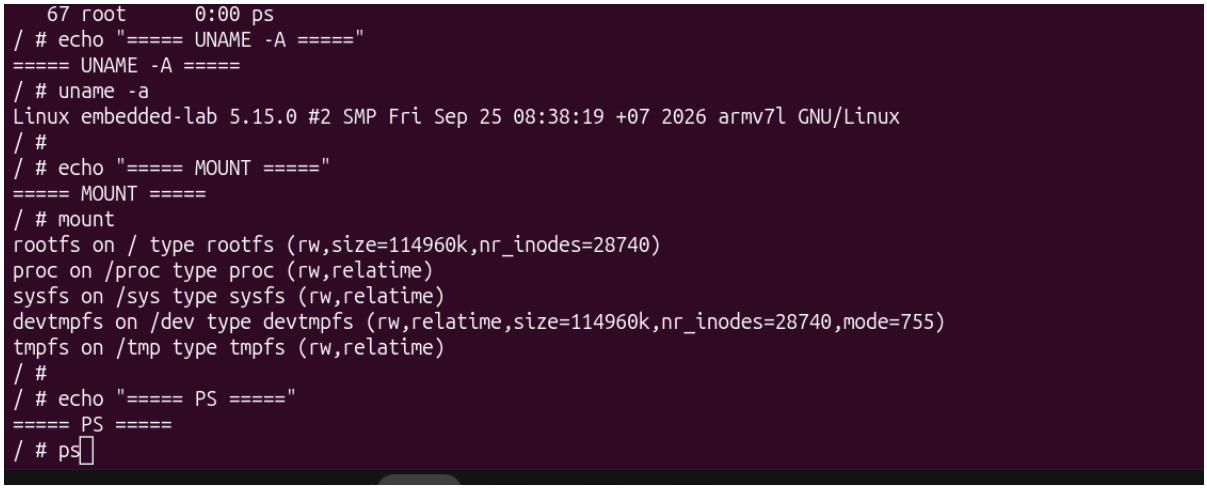


Hình 18. Các device node cần thiết được tạo trong devtmpfs

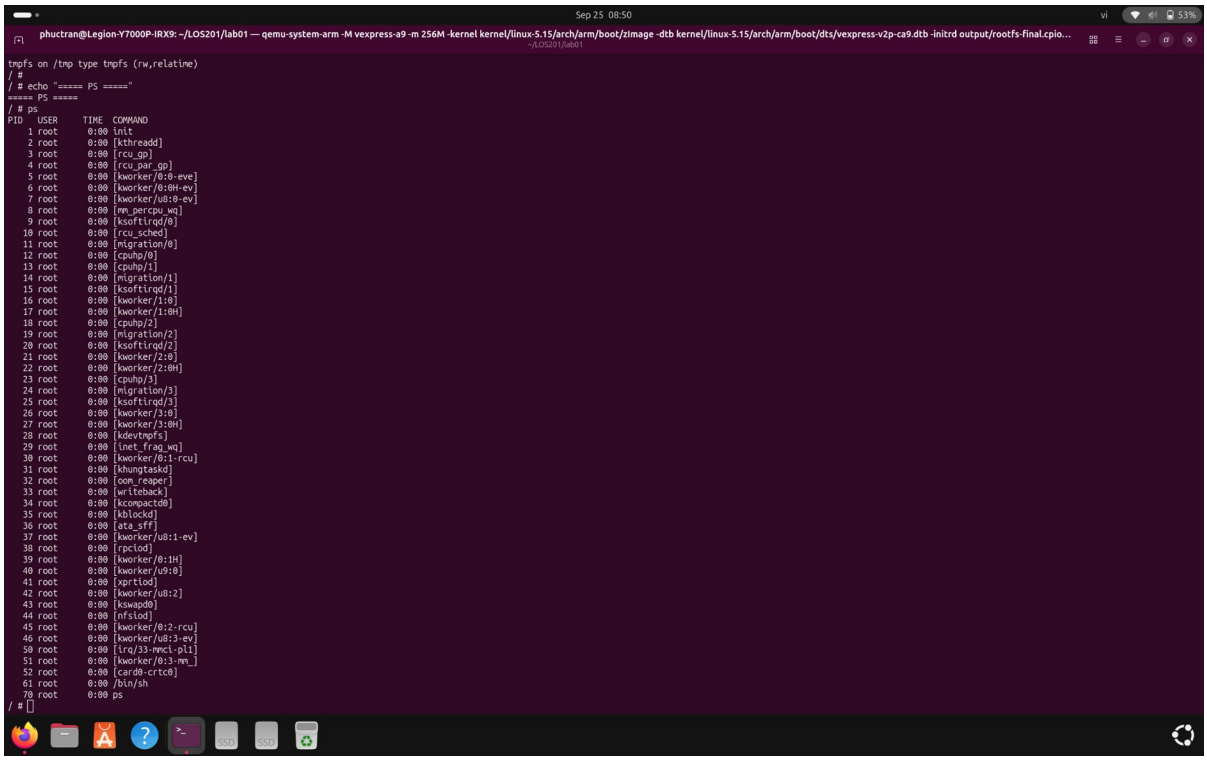


Hình 19. Xác minh rootfs cuối với hostname, SMP và tmpfs

Ba lệnh uname -a, mount và ps được chạy trực tiếp trong BusyBox shell để xác minh phiên bản kernel, kiến trúc CPU, các filesystem ảo và tiến trình init.



Hình 20. Kết quả uname -a và mount trong BusyBox



Hình 21. Kết quả ps xác nhận tiến trình init và shell

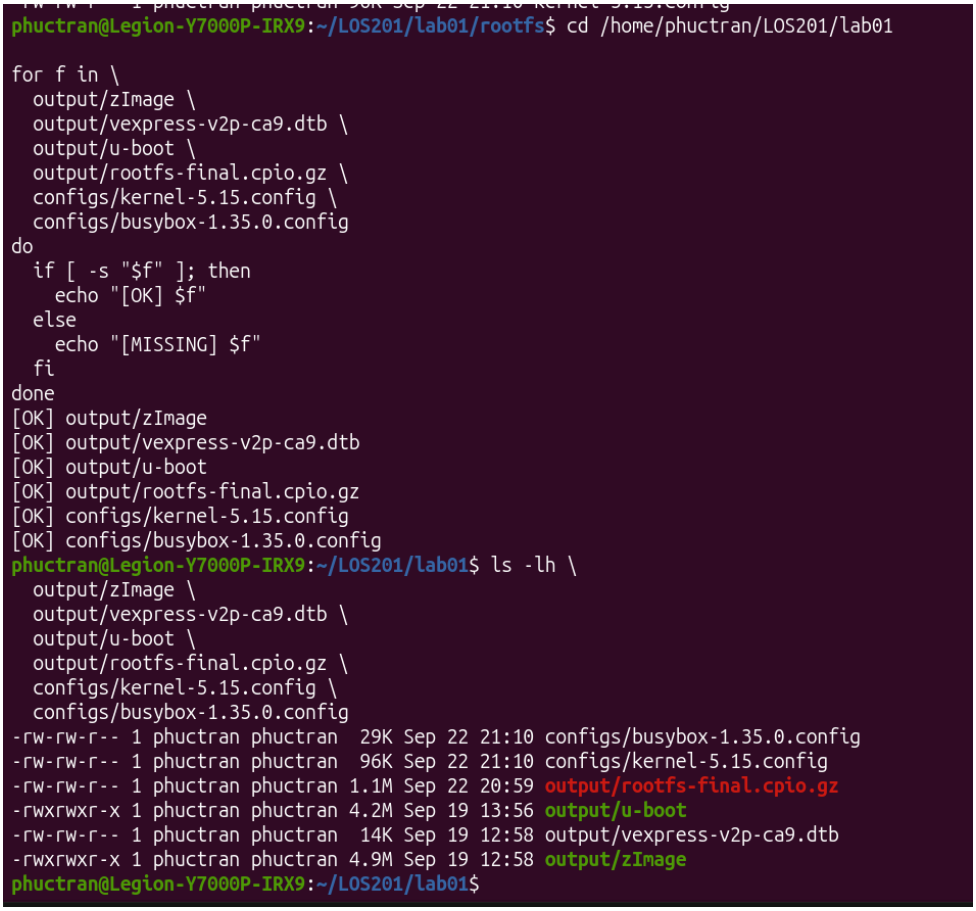
uname -a xác nhận Linux 5.15.0 trên ARMv7. mount cho thấy rootfs, proc, sysfs, devtmpfs và tmpfs đã được mount. ps xác nhận PID 1 là init, đồng thời /bin/sh và lệnh ps đang hoạt động.

6.7 Kiểm tra sản phẩm cuối

Sau khi hoàn thành boot test, các sản phẩm quan trọng được kiểm tra bằng test -s và ls -lh. Sáu mục đều tồn tại, có dữ liệu và sẵn sàng dùng cho bài nộp.

Sản phẩm	Kích thước thực tế	Trạng thái
output/zImage	4.9 MB	OK

Sản phẩm	Kích thước thực tế	Trạng thái
output/vexpress-v2p-ca9.dtb	14 KB	OK
output/u-boot	4.2 MB	OK
output/rootfs-final.cpio.gz	1.1 MB	OK
configs/kernel-5.15.config	96 KB	OK
configs/busybox-1.35.0.config	29 KB	OK



Hình 22. Các sản phẩm cuối cùng của LAB01 đều tồn tại và hợp lệ

7 Phân tích và trả lời câu hỏi

7.1 Câu 1 So sánh arm linux gnueabi và arm linux gnueabihf

Cả hai toolchain đều tạo mã máy ARM, nhưng khác nhau ở ABI dùng khi chương trình trao đổi tham số dấu phẩy động. gnueabi thường dùng soft-float ABI: tham số float và double được truyền theo quy ước của các thanh ghi số nguyên, dù trình biên dịch vẫn có thể phát sinh lệnh VFP trong một số cấu hình. gnueabihf dùng hard-float ABI: tham số dấu phẩy động được truyền trực tiếp qua các thanh ghi VFP theo chuẩn hard-float.

Cortex-A9 thuộc ARMv7-A và thường có VFPv3, nên gnueabihf phù hợp với userspace của bài lab và tận dụng phần cứng dấu phẩy động. Quan trọng nhất là toàn bộ executable, dynamic loader và thư viện userspace phải dùng cùng ABI; trộn hard-float với soft-float có thể gây lỗi liên kết, thiếu interpreter hoặc sai quy ước gọi hàm.

Nếu chỉ build Linux kernel bằng toolchain gnueabi với đúng kiến trúc ARMv7 và đúng cấu hình board, kernel không nhất thiết sẽ hỏng và vẫn có thể boot, vì kernel không liên kết với glibc và không dùng phép toán dấu phẩy động theo cách của ứng dụng. Tuy nhiên userspace soft-float sẽ không tận dụng hard-float ABI, và không thể trộn trực tiếp với thư viện armhf. Vì vậy, trong bài lab này gnueabihf được dùng để giữ target ARMv7 hard-float nhất quán từ toolchain tới BusyBox.

7.2 Câu 2 Memory map của vexpress a9

Địa chỉ bắt đầu của RAM là đặc điểm của từng thiết kế phần cứng. Trên vexpress-a9, vùng DRAM được ánh xạ từ 0x60000000; trong khi Versatile PB có cách bố trí khác và RAM bắt đầu tại 0x00000000. Địa chỉ thấp trên vexpress-a9 được dành cho các vùng thiết bị, bus và thanh ghi điều khiển theo memory map của platform.

Vì vậy bootcmd phải đặt zImage, DTB và initramfs vào các địa chỉ hợp lệ bên trong DRAM. Cấu hình của bài dùng DTB tại 0x60000000, zImage tại 0x60100000 và initramfs tại 0x64000000. Các vùng phải đủ xa để image không ghi đè lên nhau trong lúc nạp hoặc giải nén.

Nếu đặt địa chỉ ngoài RAM, trỏ vào vùng MMIO hoặc làm các image chồng lấn, U-Boot có thể báo bad magic, QEMU có thể phát sinh data abort, kernel có thể treo khi giải nén, DTB bị hỏng hoặc initramfs không còn nguyên vẹn. Khi đó hệ thống không thể đi tới userspace.

7.3 Câu 3 Giải thích MIDR 0x410fc090

MIDR là Main ID Register của ARM. Giá trị 0x410fc090 được tách theo các trường chuẩn như sau:

Trường	Bit	Giá trị	Ý nghĩa
Implementer	31:24	0x41	ARM Limited
Variant	23:20	0x0	Biến thể r0
Architecture	19:16	0xF	Kiến trúc được xác định theo các feature register
Part number	15:4	0xC09	ARM Cortex-A9
Revision	3:0	0x0	Revision p0

Như vậy giá trị cụ thể 0x410fc090 mô tả CPU do ARM thiết kế, part number Cortex-A9, variant 0 và revision 0, thường viết là r0p0. Một cấu hình QEMU khác có thể báo variant khác, nhưng part number 0xC09 vẫn xác định lõi Cortex-A9.

7.4 Câu 4 Vì sao BusyBox nên static linking

Trong initramfs tối giản, static linking giúp /bin/busybox tự chứa mã thư viện cần thiết. Kernel có thể chạy BusyBox ngay sau khi giải nén rootfs mà không cần tìm dynamic loader. Cách này giảm số file phải sao chép, tránh sai phiên bản thư viện và làm quá trình debug đơn giản hơn, đổi lại binary BusyBox lớn hơn.

Nếu build dynamic, rootfs phải có đúng dynamic loader và các shared library dành cho ARM hard-float. Tập file chính xác phụ thuộc cấu hình BusyBox; có thể kiểm tra bằng readelf -l, readelf -d hoặc công cụ cross-readelf. Những file thường gặp gồm:

File	Vị trí thường gặp trong sysroot armhf	Vai trò
ld-linux-armhf.so.3	/usr/arm-linux-gnueabihf/lib/	Dynamic loader khởi động executable
libc.so.6	/usr/arm-linux-gnueabihf/lib/	C library và system-call wrapper
libm.so.6	/usr/arm-linux-gnueabihf/lib/	Hàm toán học nếu cấu hình cần dùng
libresolv.so.2	/usr/arm-linux-gnueabihf/lib/	Hỗ trợ phân giải tên nếu applet mạng cần dùng

Các thư viện phải được đặt vào đúng đường dẫn bên trong rootfs, ví dụ /lib/arm-linux-gnueabihf hoặc đường dẫn mà interpreter ghi trong ELF mong đợi. Thiếu ld-linux-armhf.so.3 thường khiến shell báo No such file or directory dù file busybox vẫn tồn tại.

7.5 Câu 5 Kernel khởi tạo gì khi bật SMP

Với hệ thống một CPU, kernel chỉ khởi tạo boot CPU. Khi CONFIG_SMP=y và QEMU chạy -smp 2, kernel còn phải phát hiện và đánh thức CPU thứ hai, tạo vùng dữ liệu per-CPU, scheduler runqueue, timer và idle thread cho từng CPU, đồng thời cấu hình interrupt controller để hỗ trợ inter-processor interrupt. Sau đó scheduler mới có thể phân phối tiến trình giữa CPU0 và CPU1.

So với -smp 1, boot log của -smp 2 thường xuất hiện hoặc thay đổi các dòng như CPU1: ... và SMP: Total of 2 processors activated. Một dòng khác thường cho biết Brought up 1 node, 2 CPUs. Câu chữ chính xác có thể thay đổi theo phiên bản kernel, nhưng ý nghĩa là CPU thứ hai đã được đưa online và tổng số CPU hoạt động tăng từ một lên hai.

Trong bài thực hành, bằng chứng ổn định nhất là /sys/devices/system/cpu/online trả về 0-1 và /proc/cpuinfo có processor 0 cùng processor 1. Giá trị possible là 0-3 chỉ nói platform có thể hỗ trợ tối đa bốn CPU; nó không có nghĩa cả bốn CPU đang chạy.

8 Khó khăn gặp phải và cách giải quyết

Khó khăn hoặc hiện tượng	Nguyên nhân	Cách xử lý và bài học
Kernel panic khi boot zImage và DTB	Chưa cung cấp root filesystem nên kernel không thể mount /dev/ram0	Xác nhận kernel đã chạy tới bước mount root; sau đó build BusyBox, tạo initramfs và truyền bằng -initrd.
Các file /init, /etc/passwd và /etc/hostname ban đầu bị thiếu	Rootfs đầu tiên chỉ đủ để boot qua /sbin/init nhưng chưa hoàn chỉnh theo yêu cầu báo cáo	Bổ sung /init có quyền thực thi, tài khoản root, hostname và cập nhật rcS trước khi đóng gói rootfs-final.
Nano báo file đang được chỉnh sửa và có job Stopped	Một phiên nano trước bị tạm dừng bằng Ctrl+Z nên giữ trạng thái chỉnh sửa	Kiểm tra nội dung đã lưu, kết thúc job cũ bằng kill -KILL %1 rồi chạy jobs để chắc chắn không còn tiến trình treo.
possible là 0-3 nhưng online chỉ là 0-1	possible mô tả giới hạn platform, online mô tả CPU thực sự được QEMU kích hoạt	Dùng online và /proc/cpuinfo để chứng minh -smp 2; không suy luận số CPU chạy từ possible.
Build kernel báo selected processor does not support dmb, ldrex và strex	Lệnh GCC chứa đồng thời -march=armv5t và cờ assembler ARMv7-A, làm kiến trúc biên dịch không thống nhất	Dùng make V=1 để kiểm tra cờ biên dịch, thử riêng mm/vmscan.o, sau đó build lại với KCFLAGS=-march=armv7-a. Kết quả zImage is ready và Exit code 0.

9 Kết luận

LAB01 đã hoàn thành mục tiêu xây dựng và khởi động một hệ thống Linux nhúng ARMv7 tối giản trên QEMU vexpress-a9. Kernel 5.15, Device Tree, U-Boot 2022.04, BusyBox 1.35.0 và initramfs đều được tạo bằng cross-compilation. Hệ thống boot thành công tới BusyBox shell, nhận hai CPU online, tạo đầy đủ device nodes, đặt hostname embedded-lab và mount /tmp bằng tmpfs.

Qua bài lab, người thực hiện hiểu rõ ranh giới giữa bootloader, kernel và userspace. U-Boot chuẩn bị tham số và vị trí image; kernel sử dụng DTB để khởi tạo phần cứng rồi giải nén initramfs; /init và BusyBox init hoàn tất quá trình userspace. Kernel panic khi chưa có rootfs cũng cho thấy log lỗi là công cụ quan trọng để xác định chuỗi boot đã dừng tại giai đoạn nào.

10 Kiểm tra trước khi nộp

- ☑ Đã điền họ tên, MSSV, lớp/nhóm và ngày thực hiện ở trang bìa.
- ☑ Đã chèn đầy đủ ảnh minh chứng cho Phần A, Phần B và Phần C.
- ☑ Ảnh đủ rõ để đọc lệnh, kết quả và dấu nhắc Terminal.
- ☑ Đã ghi rõ file initramfs sử dụng là output/rootfs-final.cpio.gz.
- ☑ Các sản phẩm zImage, DTB, U-Boot, initramfs và file cấu hình đã được kiểm tra.
- ☑ Đã xuất bản Word sang PDF và kiểm tra lại đủ 24 trang, không có nội dung bị cắt.

11 Tài liệu tham khảo

[1] Tài liệu LAB01 Embedded Linux Systems, Cấu hình Kernel và khởi động hệ thống Linux nhúng ARMv7.

[2] Linux Kernel Documentation, https://www.kernel.org/doc/html/latest/

[3] U-Boot Documentation, https://u-boot.readthedocs.io/

[4] BusyBox Documentation, https://busybox.net/downloads/BusyBox.html

[5] QEMU ARM System Emulator Documentation,
https://www.qemu.org/docs/master/system/arm/